it in plain JavaScript, but it's considered a bad idea.) Hence, we use the workaround here to give us an asynchronous effect. There are also third-party add-ons that have written custom asynchronous versions of useEffect.[66]

Here's what happens now when the Venue component is rendered:

- The component renders with the venueData set to its initial value, an empty array.
- The function argument to useEffect is called, triggering a call to the server and eventually updating the value of the venue data.
- The updated venue data value triggers a re-render of the Venue component.

By default, that re-render would trigger another invocation of the argument to useEffect, which we don't want. We want to be more in control of the calls to the server, and we definitely don't want that setInterval to be invoked multiple times.

The second argument to useEffect, which is optional, allows us more control over when the useEffect function is invoked. There are three possible values for that second argument:

- If the second argument is not present, the effect is invoked every time the component renders.
- If the second argument is an empty array, the effect is only invoked the first time the component renders and never again.
- If the second argument is an array containing values, those values are typically values from the props array. (In our case, we might have something like [rows, concertId].) If that array contains values, the effect is only invoked on rendering if one of the values in the array changed. Under most circumstances, if you have props referenced in the useEffect, then every props value so referenced should be part of this array.

You can use this second argument for performance purposes to minimize calls to